

Spark Pessoa API

1. Visão geral

O projeto **Spark Pessoa API** consiste em uma aplicação web desenvolvida em Java utilizando o **Spark Framework**, com o objetivo de implementar uma API REST para o cadastro e gerenciamento de pessoas.

A aplicação disponibiliza operações de **CRUD (Create, Read, Update e Delete)**, permitindo cadastrar, consultar, atualizar e excluir pessoas.

Para o armazenamento dos dados, o projeto utiliza o banco de dados **H2 em memória**, enquanto a comunicação com o banco é realizada utilizando **JDBC puro**. Para o tratamento dos dados em formato JSON, é utilizada a biblioteca **Gson**.

2. Principais características

As principais tecnologias utilizadas no projeto são:

- **Java 17** — linguagem de programação utilizada no desenvolvimento;
- **Spark Java 2.9.4** — framework utilizado para criação da API REST;
- **Gson 2.11.0** — biblioteca utilizada para conversão entre objetos Java e JSON;
- **H2 Database** — banco de dados utilizado em memória;
- **JDBC** — utilizado para comunicação direta com o banco de dados;
- **Jetty** — servidor web utilizado internamente pelo Spark;
- **Maven 3.9+** — ferramenta utilizada para gerenciamento das dependências e construção do projeto.

Um ponto importante é que o **Spark Java não deve ser confundido com o Apache Spark**. Apesar de possuírem nomes semelhantes, são tecnologias diferentes e possuem objetivos distintos.

3. Arquitetura da aplicação

O projeto utiliza uma arquitetura dividida em camadas, buscando separar as responsabilidades de cada parte da aplicação.

As principais camadas são:

Model

Representa os dados da aplicação.

No projeto, a classe **Pessoa** representa uma pessoa cadastrada no sistema, contendo seus respectivos atributos.

Repository

É responsável pela comunicação com o banco de dados.

O projeto utiliza **JDBC puro**, realizando operações SQL diretamente no banco H2.

Entre as operações existentes estão:

- buscar todas as pessoas;
- buscar uma pessoa pelo ID;
- inserir uma pessoa;
- atualizar uma pessoa;
- excluir uma pessoa.

Service

É responsável pelas regras de negócio da aplicação.

Por exemplo, antes de realizar determinadas operações, o Service pode verificar se os dados recebidos são válidos.

Essa separação permite que as regras de negócio não fiquem diretamente misturadas com o código responsável pelas requisições HTTP.

Controller

É responsável por receber as requisições HTTP e retornar as respostas para o cliente.

As rotas da API são configuradas no Controller utilizando as funções disponibilizadas pelo Spark.

O fluxo básico da aplicação pode ser representado da seguinte maneira:

Cliente → Controller → Service → Repository → Banco H2

4. Vantagens do Spark Java

Uma das principais vantagens do Spark Java é sua **simplicidade**.

Por ser um microframework, ele apresenta uma estrutura relativamente pequena e permite criar endpoints HTTP sem a necessidade de uma grande quantidade de configurações.

Outro ponto positivo é a possibilidade de compreender melhor o funcionamento interno de uma aplicação web.

No projeto desenvolvido, por exemplo, várias funcionalidades que normalmente são realizadas automaticamente pelo Spring Boot precisam ser implementadas manualmente.

Isso permite observar diretamente:

- criação das rotas;
- conversão de JSON;
- validação dos dados;
- comunicação com o banco;
- tratamento das respostas HTTP;
- organização das camadas da aplicação.

O Spark também utiliza um servidor Jetty embarcado, permitindo executar a aplicação sem a necessidade de configurar manualmente um servidor externo.

Por ser um framework pequeno, também pode ser interessante para aplicações menores ou projetos acadêmicos nos quais não existe necessidade de uma estrutura muito complexa.

5. Desvantagens do Spark Java

Apesar da simplicidade ser uma vantagem, ela também pode representar uma desvantagem em projetos maiores.

Como diversas funcionalidades não são fornecidas automaticamente pelo framework, o desenvolvedor precisa implementar várias partes da aplicação manualmente.

No projeto, por exemplo, a conversão entre objetos e JSON é feita explicitamente utilizando Gson. Da mesma forma, a comunicação com o banco é realizada utilizando JDBC e comandos SQL diretamente.

Em frameworks mais completos, como o Spring Boot, existem abstrações que facilitam essas tarefas.

Outra possível desvantagem é que o Spark possui uma estrutura mais enxuta e menos abrangente, fazendo com que determinadas funcionalidades precisem ser desenvolvidas ou configuradas manualmente.

Isso pode aumentar a quantidade de código necessário conforme a aplicação cresce.

Portanto, o Spark pode ser bastante interessante para projetos pequenos e para fins

educacionais, mas pode exigir mais trabalho manual em aplicações maiores e mais complexas.

6. Servidores Web disponíveis

No projeto, o servidor utilizado é o **Jetty**, que é executado de maneira embarcada pelo Spark Framework.

Isso significa que não é necessário instalar e configurar separadamente um servidor web para executar a aplicação.

O projeto configura a aplicação para utilizar a porta:

4567

Dessa forma, depois de iniciar o programa, a API pode ser acessada localmente por meio do endereço:

<http://localhost:4567>

O README também faz uma comparação com aplicações desenvolvidas utilizando Spring Boot, nas quais normalmente é utilizado o servidor Tomcat embarcado.

Assim, no contexto do projeto:

Spark Java → Jetty

Spring Boot → Tomcat

7. Configurações necessárias para executar a aplicação

Para executar o projeto são necessários alguns softwares e configurações.

Os principais requisitos são:

- **Java 17 ou superior;**
- **Maven 3.9 ou superior;**
- **IntelliJ IDEA** para desenvolvimento;
- **Postman** para realizar testes das requisições da API.

Também é necessário que as dependências do projeto sejam baixadas pelo Maven.

Depois de abrir o projeto na IDE, é necessário permitir que o Maven carregue as dependências especificadas no arquivo `pom.xml`.

Para iniciar a aplicação, deve-se executar a classe:

App.java

A aplicação inicia o servidor Jetty utilizando a porta **4567**.

O banco de dados utilizado é o H2 em memória. A configuração utilizada no projeto é:

jdbc:h2:mem;DB_CLOSE_DELAY=-1

Por ser um banco em memória, os dados são temporários e são perdidos quando a aplicação é encerrada.

8. Endpoints disponibilizados

A API disponibiliza os seguintes endpoints:

GET /pessoas

Retorna todas as pessoas cadastradas.

GET /pessoas/

Retorna uma pessoa específica utilizando seu ID.

POST /pessoas

Realiza o cadastro de uma nova pessoa.

Os dados são enviados no corpo da requisição utilizando o formato JSON.

PUT /pessoas/

Atualiza os dados de uma pessoa existente.

DELETE /pessoas/

Exclui uma pessoa utilizando seu ID.

Esses endpoints permitem realizar as operações básicas de um CRUD.

9. Responsáveis pelo desenvolvimento

10. Conclusões sobre o uso do framework

A utilização do Spark Java no desenvolvimento deste projeto foi interessante principalmente por permitir compreender conceitos que normalmente ficam escondidos atrás das abstrações de frameworks mais completos.

Um dos principais pontos positivos foi a simplicidade para criar as rotas da API. Com poucas configurações, foi possível iniciar um servidor HTTP e disponibilizar os endpoints necessários para realizar o CRUD de pessoas.

Por outro lado, essa simplicidade também faz com que algumas tarefas precisem ser realizadas manualmente. A conversão de objetos para JSON, a comunicação com o banco utilizando JDBC e algumas validações são responsabilidades que ficam mais próximas do desenvolvedor.

Em relação à configuração, o framework apresenta uma estrutura relativamente simples. O servidor Jetty é embarcado e o projeto pode ser executado diretamente pela aplicação Java, sem a necessidade de instalar um servidor externo.

Em relação à aprendizagem, o Spark Java pode ser interessante para estudantes porque permite visualizar de maneira mais direta como uma aplicação web funciona. Ao comparar o projeto com uma aplicação Spring Boot, é possível perceber quais funcionalidades são fornecidas automaticamente por frameworks maiores.

Entretanto, para aplicações maiores, a quantidade de tarefas manuais pode se tornar uma desvantagem.

Portanto, a conclusão é que o **Spark Java é uma opção interessante para projetos pequenos, APIs simples**, pois possui uma estrutura enxuta e permite compreender conceitos fundamentais de aplicações web.

Para projetos maiores e com maior quantidade de funcionalidades, pode-se utilizar frameworks mais completos, como o Spring Boot, que oferecem diversas abstrações e ferramentas prontas.